ORVESSA: HOW FAR DOES A SMALL LLM LEARN A LANGUAGE IT HAS NEVER SEEN?

Kavinraj
Group 14, roll number 24IM10055
Indian Institute of Technology Kharagpur
Course project, AI60213 Foundations of Large Language Models
Code: `https://github.com/Lsidvd234/alien-rosetta`

ABSTRACT

Studying how a language model learns a new language is hard with real languages: the model may have seen them in pretraining, and there is no exact answer key. We build a generator that invents a complete Tamil-like language (subject–object–verb order, case and agreement endings, sound changes, 11,078 root words) written in 27 invented glyphs that no tokenizer has seen, with a deterministic translation for every English sentence. We fine-tune Qwen3-0.6B-Base with LoRA to translate English into it, first writing a structured *meaning analysis* and then the glyphs, so every error can be split into a misreading of the English or a mistake in the new language. Across 165 training runs on free GPUs we find: (A) generalisation stops at what training covers: held-out English structures score 0.00–0.09, while putting back only 50 passive sentences (0.06% of the data) lifts passive reading from 0.00 to 0.53–0.66; (B) a spelling rule transfers to never-seen words when it covers the whole vocabulary (0.67) but is learned mostly as a list when it covers a 4–12% minority (0.19–0.27); (C) on real English only more varied English helps (Tatoeba meaning 0.62 $\rightarrow$ 0.92, 2 seeds), while focal loss, paraphrases, more epochs, decoding tricks and a 3$\times$ larger model do not; (D) on real English the failures are in *reading* the English, and where the English has words the training never had, in both reading and writing; probes and steering show that the output is written from the model's English meaning representation, where most errors already sit; (E) in arithmetic written in glyph digits, a fact removed from training is rebuilt only from facts the model has seen in the same role, and five examples of that role are enough.

1 INTRODUCTION

Large language models are increasingly adapted to low-resource languages, new notations and domain scripts, often with a small model and a parameter-efficient method such as LoRA (Hu et al., 2022). What such a model can and cannot learn from a modest amount of parallel data is therefore a practical question. It is also a hard one to answer cleanly with real languages: pretraining may already contain the language, the gold translations are themselves noisy, and one cannot change a single property of a real language and keep everything else fixed.

We take the opposite route and *invent* the language. A generator produces a language with a fixed grammar, vocabulary and script, renders any meaning into it deterministically, and lets us change one property at a time: which sentence structures appear in training, which words are spelled by a rule, how much real English is mixed in. The model has certainly never seen the language, and every test sentence has exactly one right answer. Our research question is:

What does it take for a small LLM to learn a language it has never seen, and where does that learning stop?

Setting: The input is an English sentence; the output is the same sentence in the invented language, written in 27 Unicode Private Use Area glyphs, each added to the tokenizer as one new token. The

main models first write a meaning analysis such as `S=NAME.Tom V=beat.FUT O=PRON.2sg` and then the glyphs. Because rendering a meaning is deterministic, any wrong output is either a *reading* error (wrong analysis) or a *writing* error (right analysis, wrong glyphs). Models are tested on generated sentences, on 600 real sentences from Tatoeba (Tatoeba contributors, 2026), and on a *sealed* set of free English written by the author and frozen before it could influence any design decision.

Contributions:

- **A controlled testbed**: an invented-language generator with an exact answer key, a meaning-first output that splits every error into reading vs writing, and a sealed test set used only as totals (Section 4).
- **Five findings with stated strength** (Section 5): (A) coverage limits structure, with a 50-example dose that switches passives on; (B) rule vs list spelling depends on how much of the vocabulary the rule covers; (C) only English variety helps real English, tested against eight alternatives; (D) on real English reading fails, and with untrained words both fail, with probes and steering locating the errors in the English representation; (E) in maths, a missing fact is rebuilt only from facts seen in the same role.
- **Pre-registered predictions**: most extension runs had their predictions and decision rules written before they ran, and results that missed a prediction are reported as such.

Every number states how many training runs it rests on: *1 run*, *2 seeds*, or a range over runs.

2 Related work

Compositional and length generalisation. Sequence models fail on held-out combinations and longer inputs (Lake & Baroni, 2018; Kim & Linzen, 2020), and transformers solve multi-step tasks by matching seen subgraphs rather than composing them (Dziri et al., 2023). Fine-tuned LLMs struggle with typologically novel features of a new language (Marmonier et al., 2025), and learning a language from a grammar book is far from solved (Tanzer et al., 2024). Our claim A confirms the general limit in a setting where coverage can be changed one structure at a time and measured as a dose.

Rules vs memorisation. Whether a pattern is learned as a rule depends on how regular it is: the Tolerance Principle (Yang, 2016; Schuler et al., 2016), neural inflection models (Kirov & Cotterell, 2018; McCurdy et al., 2020), and compositional languages in neural agents (Galke et al., 2024); Friedman et al. (2026) find a gradual rather than sharp change in a small LM. Claim B is a controlled same-model contrast between the two ends; we do not claim a threshold.

Synthetic vs real input. Semantic parsers trained on synthetic utterances fail on real ones (Wang et al., 2015; Herzig & Berant, 2019; Marzoev et al., 2020), and scale does not close out-of-distribution gaps (Qiu et al., 2022). In low-resource translation the *generation* side is usually the barrier (Bafna et al., 2025; Vamvas et al., 2026); our claim D finds the opposite because here the target language is dense in training and the English input is the unfamiliar side.

Probing and steering. Linear probes read features from hidden states (Alain & Bengio, 2016); adding activation differences steers outputs (Turner et al., 2023; Panickssery et al., 2024), including tense (Klerings et al., 2025); we use both as evidence for claim D, in a language learned from scratch with LoRA. **Facts in a new role.** One stored fact does not give its reverse (the reversal curse, Berglund et al., 2024); claim E asks when a missing fact can be rebuilt from facts the model has.

3 Problem formulation

The language: A generator defines a meaning space $\mathcal{M}$, an English renderer E and a glyph renderer R. A meaning $m \in \mathcal{M}$ is a list of events, each a set of fields (doer `S`, verb with tense, aspect, modality, negation and question marks `V`, object `O`, place, manner, time, 21 relation fields, …; Appendix A). $E(m, v)$ renders m as English using a surface variant v (word order, voice, contractions, casing, typos, synonyms). $R(m)$ is deterministic: it orders the words subject–object–verb, adds case

Table 1: Training and test format: English input, meaning analysis (the model's first output), and the glyph output, shown schematically (the glyphs are Private Use Area characters). Real Tatoeba sentences.

English input x	Meaning analysis m
Yanni washed the figs.	`S=NAME.Yanni V=wash.PAST O=fig.PL`
You guys can't scare me.	`S=PRON.2pl V=scare.PRES.CAN.NEG O=PRON.1sg`
Tom is going to beat you.	`S=NAME.Tom V=beat.FUT O=PRON.2sg`
Doesn't Tom ever wear a suit?	`S=NAME.Tom V=wear.PRES.NEG.Q M=ever O=suit`

Glyphs $y = R(m)$ for the third row, glossed: *Tom you-ACC beat-FUT-3sg.m* (subject–object–verb order, case and agreement endings, sound changes at the joins).

and agreement endings, applies sound changes where pieces join, and spells every sound in glyphs from the alphabet Σ, $|\Sigma| = 27$.

Input, process, output: The input is an English sentence $x = E(m, v)$. The model outputs the target string

$$z = m \parallel \text{=>} \parallel y, \qquad y = R(m), \tag{1}$$

("analysis first"), or $z = y$ for direct models. The model is a causal LM p_θ with frozen pretrained weights W, LoRA updates $\Delta W = BA$ with $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times d}$, $r = 16$, on every attention and MLP projection, and a trainable embedding table $E_\Sigma \in \mathbb{R}^{|\Sigma|\times d}$ for the new glyph tokens (tied input and output). Training minimises the token-level cross-entropy on the target only:

$$\mathcal{L}(\theta) = -\sum_{(x,z)\in\mathcal{D}} \sum_{t=1}^{|z|} \log p_\theta(z_t \mid x, z_{<t}), \qquad \theta = \{A, B, E_\Sigma\}. \tag{2}$$

Each glyph g spells one sound $s(g)$ (*k*, *zh*, *aa*, ...); its embedding starts at the embedding of the token for that sound, $E_\Sigma[g] \leftarrow e_{\mathrm{tok}(s(g))}$ (the sound with no letter, the virama, starts at the mean embedding).

Reading and writing errors: For a test pair (x, m, y) and output $(\hat{m}, \hat{y})$ we define

$$\text{reading error: } \hat{m} \neq m; \qquad \text{writing error: } \hat{m} = m \wedge \hat{y} \neq y. \tag{3}$$

Because R is deterministic, a writing error is a true error in the new language, and for a reading error we can also check whether $\hat{y} = R(\hat{m})$, i.e. the model faithfully wrote its own misreading.

Coverage: For a training set $\mathcal{D}$ let $c_\mathcal{D}(k)$ count the occurrences of a meaning key k (a word, a structure feature, a multiplication fact). The questions of this paper are how accuracy on test items depends on $c_\mathcal{D}$ of the keys they need: zero (held out), small (a dose), or large; and which changes to $\mathcal{D}$ or to the model move accuracy on real English, where $c_\mathcal{D}$ is not under our control.

4 Experimental setup

Data pipeline: The generator samples a meaning *first* (so labels are right by construction) from *families*: kind of utterance (statement, yes/no and wh-question, command, fragment, greeting), predicate, aspect, modality, subject and object kinds, determiners and owners, relation and time fields, joining words; then a surface variant. Vocabulary is drawn by tempered English word frequency, with a *word floor* guaranteeing every word at least 12 sightings. A main training set has 80,000 translation pairs plus 2,500 refusal examples (non-English input $\rightarrow$ a refusal output) and 6,000 replay examples (88,500 in total). Extensions change one thing: hold structures out, put a dose back, spell loanwords by a sound rule, replace 15% of the pairs with 12,269 real Tatoeba sentences labelled by an automatic English reader, add the everyday English the generator left out, or paraphrase training English with a frozen instruct model. Figure 1 shows the format.

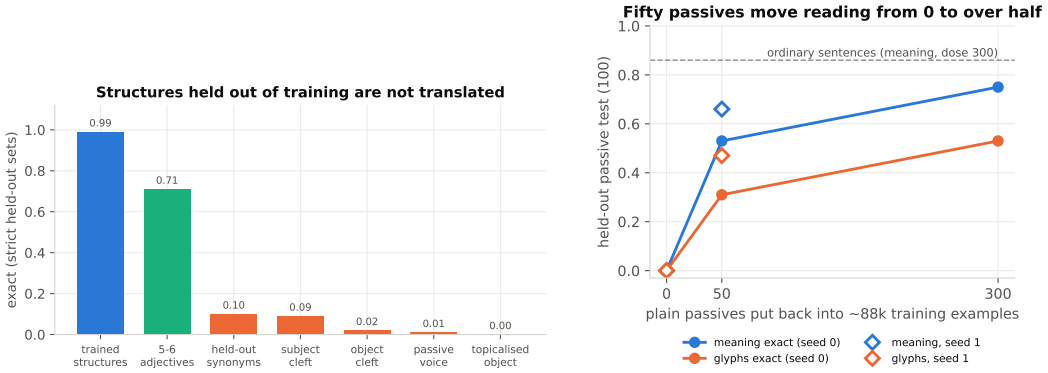


Figure 1: Left: English structures held out of training (first language version, analysis-first model, 5 epochs; three runs give 0.00–0.09 on every held-out structure). Right: passives put back as a dose, nothing else changed (1 run per dose; diamonds: second training seed).

Model and training: Qwen3-0.6B-Base (Yang et al., 2025) (a 1.7B model was used once), LoRA rank 16, $\alpha = 32$, no dropout, on q, k, v, o, gate, up and down projections; the 27 glyph tokens added with trainable embeddings; AdamW, learning rate 2×10^{-4}, no weight decay, 3% linear warm-up then linear decay, effective batch 32, 4 epochs (11,064 steps), fp16. One run takes about 6 GPU-hours on a free Kaggle T4. The baseline for every extension is the analysis-first model with no structures held out (`e2b-v2fix-cot-80k-noheld`).

Maths extension: The same recipe on arithmetic in base 7, written with 7 glyph digits and operator glyphs (new tokens that start at the real digits and operators): 60k mixed examples of addition, multiplication and base conversion, 2 epochs; trained sizes up to 3+3 digits and 2×2 (later 3×2) digits. Optional *scratchpads* write the long-multiplication working before the answer. To remove a fact such as 5×6, every training problem whose working needs it is dropped (about 9% of multiplications).

Test sets: (1) *Generated*: 300 sentences from the generator, never in training, shared by all runs (paired tests are valid), plus 100-sentence held-out-structure sets. (2) *Tatoeba 600*: real sentences filtered to the language's scope; gold from the automatic reader. A blind hand-check of 70 gold labels found only about 0.6 right, so Tatoeba measures agreement with the reader. (3) *Sealed*: 147 in-scope and 25 out-of-scope free sentences written by the author, annotated by hand, frozen with a SHA-256 fingerprint before any reader-labelled training data existed, and used *only as totals* (no sentence was inspected after freezing).

Metrics and statistics: Meaning exact, glyphs exact, word accuracy, refusal rate. Paired McNemar tests (McNemar, 1947) on the same sentences (for the sealed set, only fix and break counts), bootstrap intervals, and a Holm correction (Holm, 1979) across 21 claim-level tests. For most extension runs the prediction and the decision rule were written in the project log before the run or before scoring; we say where a result was found after the data.

5 Results

5.1 A. Generalisation stops at what training covers

Holding 20% of English structure variants out of training gives near-zero accuracy on exactly those structures (passive 0.00–0.02, clefts 0.01–0.09, topicalised objects 0.00–0.01, three runs) while trained structures stay at 0.97–0.99 (Figure 1, left). The misreading is by position: a passive takes its first noun as the doer. A dose test isolates the cause in the final language: with only plain passives held out and 0, 50 or 300 put back into ∼88k examples, held-out passive meaning goes 0.00 / 0.53 / 0.75, and the 0 → 50 step replicates on a second seed (0.00 / 0.66) (Figure 1, right). Fifty examples, 0.06% of the data, switch the structure on; the rest of the language is unchanged.

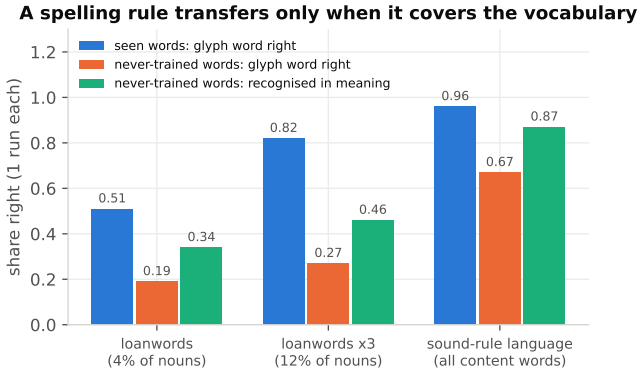


Figure 2: Words spelled by fixed sound rules from their English letters. Loanwords: only a minority of nouns follow the rule (rate 0.04, and 0.12 in the "×3" run). Sound-rule language: every content word does, and 20% of content words (2,232) never appear in training. 1 run per condition, 300–600 generated sentences.

The same limit appears in arithmetic written in glyph digits (base 7). Removing every training multiplication that needs one single-digit fact drops problems that need it to 0.05 (5×6), 0.10 (3×4) and 0.41 (2×2), while problems that avoid it stay at 0.91–0.96; a control that also removes the swapped twin keeps the avoid-group at 0.82–0.89. A bridge run that trains base conversion of every product, including the removed one, still does not rebuild 5×6 (problems needing it 0.03). A scratchpad trained on workings with at most 3 steps per row writes exactly 2 rows of 3 steps on every longer problem (1,798 of 1,800 rows; exact 0.00–0.01): it learned the template of its longest working, not the loop, the subgraph-matching pattern of Dziri et al. (2023).

Strength: strong as a description (many runs, two seeds where tested); causal for one structure and one fact. New relative to prior work: the 50-example dose and the fixed scratchpad shape with dropped-digit answers. Training much longer does not remove the limit either: with weight decay 1.0 and up to 60k steps we saw no grokking (no late jump in test accuracy) in translation or maths (Appendix E).

5.2 B. A SPELLING RULE TRANSFERS ONLY WHEN IT COVERS THE VOCABULARY

Two extensions spell words by the same letter-to-sound rules (*b*→*p*, *th*→*t*, …). If the rule were learned, words never seen in training should be spelled almost as well as seen ones. When only loanwords follow it (4% of nouns, marked `LOAN`), new loans are spelled right 0.19 against 0.51 for seen loans; tripling the loan rate lifts seen loans to 0.82 but new loans only to 0.27, missing the pre-registered +0.10, and the seen–new gap grows from 0.32 to 0.55 (Figure 2). Some of the rule is learned: one-letter mappings transfer better than multi-letter ones (0.23 vs 0.12, Fisher $p = 0.005$), and near-miss spellings rise from 21% to 33% of errors. Recognition fails first: only 0.34–0.46 of new loans are marked as loans; most are written as a bare English word the language does not have. When *every* content word follows the rule, never-trained words are spelled right 0.67 (seen words 0.96 in the same sentences) and recognised 0.87, at no cost elsewhere (generated glyphs 0.71 vs 0.61 baseline).

Strength: moderate (1 run per condition). The two conditions also differ in what must be recognised, and the general idea is known (Tolerance Principle); we claim only the two ends, not a threshold.

5.3 C. ONLY ENGLISH VARIETY HELPS REAL ENGLISH

Trained models read generated English almost perfectly (0.943 of sentences with every meaning fact right), but real English is far harder: Tatoeba meaning 0.62, sealed 0.47. Replacing 15% of training with real Tatoeba sentences lifts Tatoeba meaning to 0.923 / 0.928 (2 seeds; +0.307, 189 fixes vs 5 breaks, $p = 1.8 \times 10^{-49}$) and sealed meaning from 0.476 / 0.463 to 0.558 / 0.558 (paired McNemar

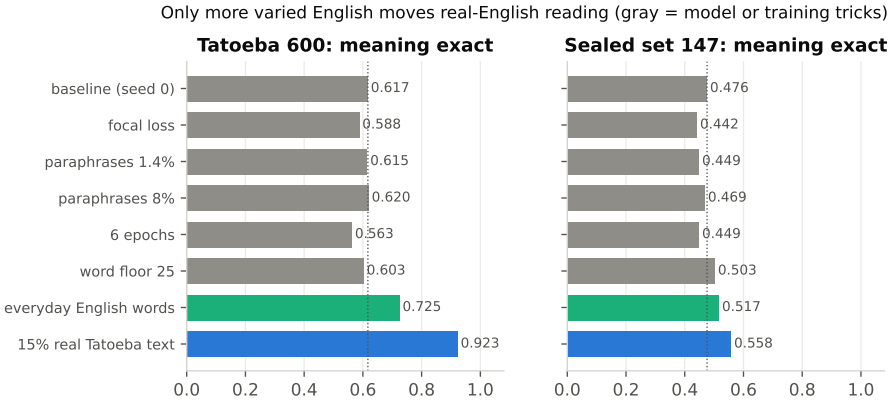


Figure 3: One change per run against the baseline. Gray: model, loss and training-length changes; green: the common English words and uses the generator left out (no reader labels); blue: 15% real Tatoeba sentences. Sealed numbers are totals over 147 sentences.

$p = 0.036$ / 0.013 per seed; not significant after the Holm correction). Because Tatoeba gold and the real training labels come from the same automatic reader, and the hand-check finds the gold right in 0.61 of the gained sentences, about +0.19 of the +0.31 rests on correct labels. A run that adds only the everyday English words and uses the generator never produced (main-verb *have/do*, *people*, *just*, nouns made from verbs; no real sentences, no reader labels) gains +0.11 on Tatoeba (77 fixes vs 12 breaks, $p < 10^{-11}$) and +0.04 on the sealed set (1 run).

Every alternative tried is null or harmful on real English (Figure 3): focal loss (-0.028, $p = 0.014$), paraphrased training English at 1.4% and 8% of the data, a word floor of 25, 6 epochs instead of 4 (-0.053, $p = 0.0002$, also after Holm: more epochs fit the generated style), a two-stage rewriter, beam search and majority voting over samples (changes below 0.01), and Qwen3-1.7B (Tatoeba reading failures 0.37 vs 0.38 at the same 2 epochs; the pre-registered $\geq$0.05 improvement fails). Of the baseline's 217 never-seen word occurrences on Tatoeba, most are everyday words and uses, not rare words; real text cuts them to 20.

Strength: strong on Tatoeba (2 seeds), **moderate on the sealed set**. The synthetic-to-real gap is known (Herzig & Berant, 2019); our contribution is a controlled test against eight alternatives.

5.4 D. On real English, reading fails; with untrained words, both

Judged on the model's own meaning, the baseline's writing errors stay roughly flat from generated to Tatoeba to sealed sentences (0.24, 0.31, 0.26) while reading errors climb (0.21, 0.38, 0.52) (Figure 4, left). On held-out English structures 73–75% of items are misread and only 0.4–1.6% miswritten. To separate the two causally we trained a reader alone (English $\rightarrow$ meaning) and a writer alone (meaning $\rightarrow$ glyphs). The reader is wrong in 74% (Tatoeba) and 90% (sealed) of pipeline failures; on held-out passives the reader scores 0.00 while the writer, given the gold meaning, writes 0.72. But the writer from gold is right only 0.63 on Tatoeba. A per-sentence test with predictions written beforehand shows why (Figure 4, right): the writer is right 0.84 when every word key of the meaning was trained at least 25 times, 0.82 when the rarest was seen 1–24 times, and 0.20 when one was never trained (a third of Tatoeba); the reader also fails there (0.25). The sealed set goes the same way (0.93 vs 0.34). For passives the mechanism is in the representation: a linear probe cannot read the passive doer from the hidden states of the model trained without passives beyond what the surface words give (0.746 vs 0.704), but reads it in models trained on passives (0.871, 0.950).

Inside the model: Why does reading, rather than writing, fail? Linear probes read tense, negation, question, aspect, modality and subject from the hidden states of every model at 0.90–1.00. Steering shows that the output is written from this English-side representation: adding the tense direction flips the output tense in 0.97–1.00 of sentences and negation in 0.91–0.99, against $\leq$0.02 for random directions, on four models (two training seeds each of the baseline and the real-text model; Figure 5),

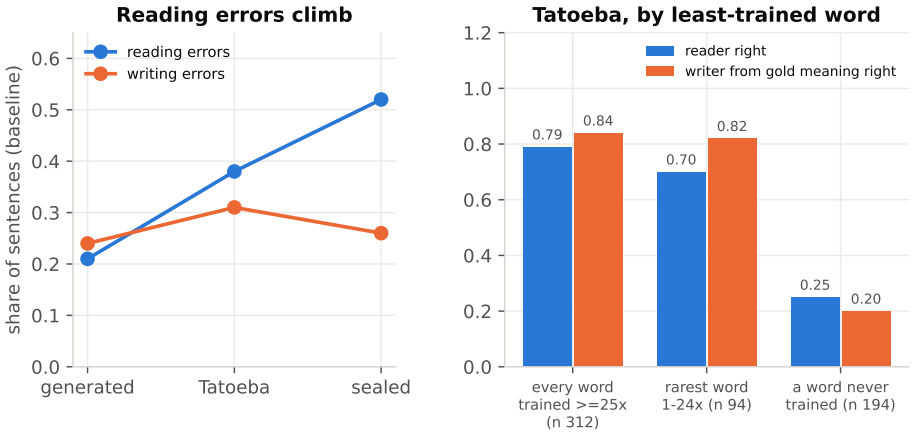


Figure 4: Left: share of sentences with a reading error ($\hat{m} \neq m$) and a writing error (judged on the model's own meaning) for the baseline as the English moves away from training. Right: separate reader and writer models, the writer given the gold meaning, on Tatoeba by the least-trained word key in the meaning (1 run each).

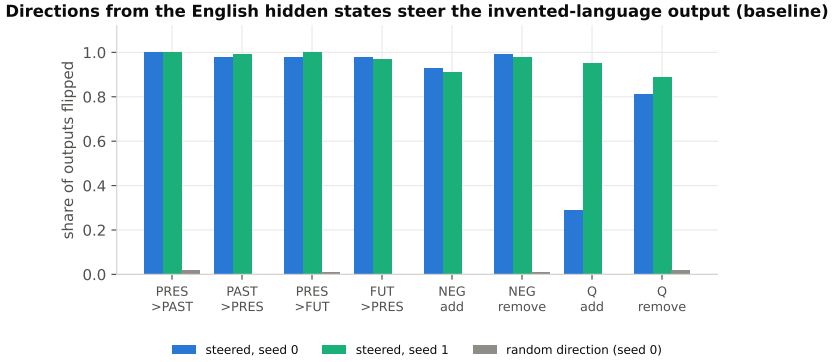


Figure 5: Steering the baseline: a mean-difference direction (from English hidden states of sentences with the two values) is added at layer 6 while the model reads the English. Flip: the output's meaning now carries the target value. Random: a same-size random direction. 100 generated sentences per pair.

and subject pushes repair 11 of 12 and 10 of 10 real Tatoeba subject errors (random or shuffled-label pushes repair at most 3). Where the output is wrong on Tatoeba, the probe is usually wrong too (right 0.41, below the majority-class rate 0.79) and in 59% of cases reads the same wrong value the output wrote: most real-English errors are already in the English representation, and the writer faithfully renders them. Tense steering itself is known (Klerings et al., 2025); here it serves as evidence for where the errors sit.

Strength: moderate (1 run per model for the reader/writer split, 2 seeds for probes and steering; the split was examined after the first results, and the later predictions were written before their runs). The usual low-resource finding is the opposite (Bafna et al., 2025); here the target language is dense in training and the English input is the unfamiliar side.

5.5 E. In maths, a missing fact is rebuilt only from facts seen in the same role

Claim A showed that a model trained without 5×6 cannot infer it. We then asked what it takes to make it *build* the missing fact, in four steps (all with 5×6 and 6×5 absent from training, checked).

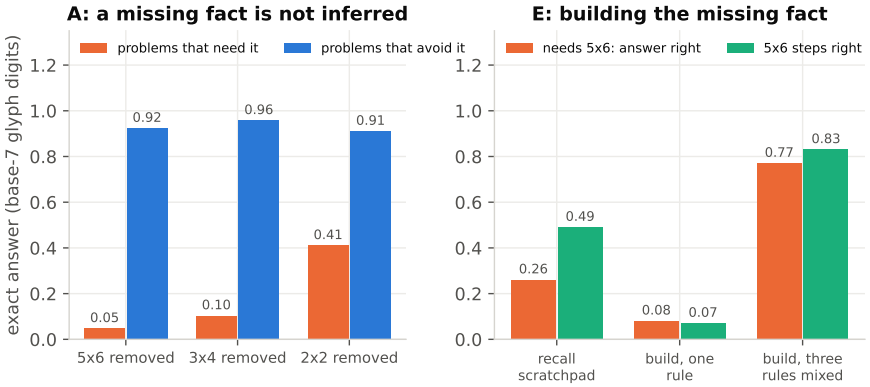


Figure 6: Arithmetic in base-7 glyph digits. Left: removing every training problem that needs one fact (claim A). Right: three scratchpads with 5×6 and 6×5 removed from training.

1. Recall the facts in a scratchpad (`5x6+0=42`): every other step of the working is right (857 of 857), but the written 5×6 steps are right only 0.49, and problems needing them 0.26 (Figure 6, right). The model guesses: the same problem gets the same 5×6 value only 60% of the time, and a vote over 8 sampled workings does not help (0.27).

2. Build each product from a smaller one, with one rule (`5x6=5x4=26+5=34+5=42`): the model follows the procedure and adds correctly, but problems needing 5×6 drop to 0.08. It writes the block wrong (`5x6=5x4=22+...`) although it computes 5×4 = 26 correctly as an ordinary step: with one rule, 5×4 appeared *as a building block* only inside the removed 5×6 working.

3. Mix three building rules, so the same facts also appear as blocks in other workings: problems needing 5×6 rise to 0.77 and its steps to 0.83. Of the 5×6 steps, 80% are built from a block seen in that role (5×4, 6×3, 6×4), and these are right 99.6% of the time; the other 20% use a block never seen in that role (4×6, 5×5) and are right 17%. The same recipe rebuilds other removed facts: 5×6 on a second seed 0.72, 3×4 0.98, 6×6 0.64 (Figure 7, right); with the whole large end of the 6 row removed it recovers 0.57–0.72 of four facts and 0.00 of 6×5. Forcing every possible route on every problem, 15 of 18 blocks follow the rule (seen as a block: right; never seen as a block: wrong), and a failing route writes a neighbour's value in the block slot (the 5×5 slot gets 26, the value of 5×4).

4. How many examples of the role are needed? Keeping one building rule, 5×4 is shown as a block in N training workings. The written 5×6 steps are right 0.00 / 0.91 / 1.00 / 0.96 / 0.93 at N = 0 / 5 / 25 / 100 / 400 (Figure 7, left): the curve is a step, not a slope, and five examples are enough. The dose never reaches the other order: 6×5, built from a block the dose does not touch, stays at 0.06 even at the largest dose (593 workings). Order alone is different: with only 6×5 removed and 5×6 still in training, the plain recall scratchpad writes the never-seen 6×5 step right in 172 of 174 cases.

So the model can compute a fact it never saw, but only by recombining facts it has already used *in the same role*; a fact it knows only in another role is not reused, which links coverage (claim A) to how facts are stored. Caveat: each condition is one training run, and same-seed maths runs vary by about ±0.1–0.2 (GPU non-determinism), so single values are approximate; the second seed is less clean (seen-role blocks 0.84, never-seen 0.56).

6 Limitations

- **Run counts.** Most extension results are single training runs; real text, steering and the passive 0/50 dose have two seeds. On the 147-sentence sealed set only differences above about 0.11 are clear for a single comparison, and the Tatoeba gold is the automatic reader's (about a third wrong by a 70-sentence hand-check).
- **GPU non-determinism.** Same-seed maths re-runs with byte-identical data differ by ±0.1–0.2 on the needs-pair tests (non-deterministic attention backward pass). A

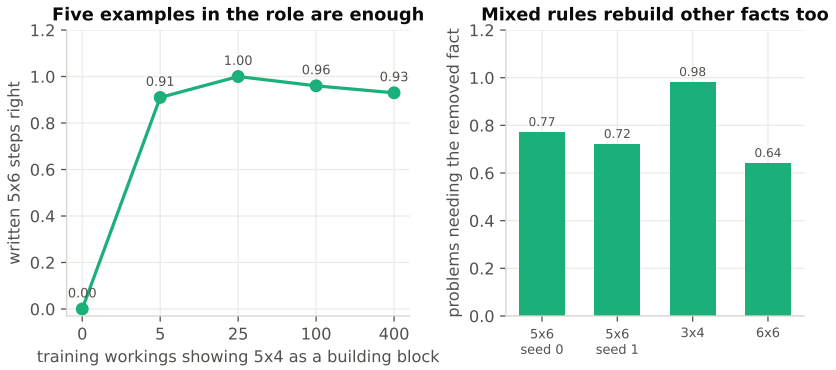


Figure 7: Left: one building rule, with 5×4 shown as a building block in N training workings. Right: the mixed-rule scratchpad with other facts removed and a second seed.

`-deterministic` option makes full language runs reproduce exactly on two different machines; all maths numbers are single draws.

- **One model family and size** (Qwen3-0.6B, one 1.7B run) and one invented grammar; claims B and D were partly formed after the first data, and are marked as such.
- **Sealed set.** Its gold meanings are reader output with hand corrections, so a model trained on reader labels can partly gain by learning reader conventions; the everyday-words run, which uses no reader labels, is the cleaner evidence for claim C.

7 Future work

The following extend experiments that already have results; their predictions are written but they were not run for this report.

- **Seed replicates for claim B**: a second training seed of the sound-rule language (does 0.67 for never-trained words replicate?).
- **Intermediate rule coverage for claim B**: languages in which 25% and 50% of content words are rule-spelled, to see whether the change between a 4–12% minority (0.19–0.27) and the whole vocabulary (0.67) is gradual or sharp, as Friedman et al. (2026) would predict for gradual.
- **Three deterministic seeds for claim E**: the mixed-rule scratchpads (5×6, 3×4, 6×6, whole 6 row removed) and the 0- and 5-workings role doses, each with three seeds, judged on 3-seed means against the written thresholds (seen-role blocks $\geq$0.90, never-seen-role blocks $\leq$0.30).
- **Determinism for maths**: confirming that the attention fix makes maths runs bit-identical, so that single-run thresholds in claims A and E can be re-judged without seed noise.

8 Conclusion

A small model learns an invented language with its own script well, but it generalises only as far as its training covers: unseen structures fail and fifty examples switch one on; a spelling rule transfers when it covers the vocabulary and stays a list when it covers a minority. On real English the limit is reading the English, not writing the new language, and only more varied English in the training data moves it; loss functions, decoding, more epochs and a 3× larger model do not. Inside the model, the invented-language output is driven by the model's English meaning representation, and most real-English errors are already present there. In arithmetic, a missing fact can be rebuilt, but only from facts seen in the same role. For anyone adapting a small model to a new language, the practical lesson is to spend effort on the coverage and variety of the source-language side of the data.

ACKNOWLEDGMENTS

Compute: free Kaggle GPUs (three accounts, two lent by friends), a laptop RTX 2050, and one paid cloud session of about USD 1.60. Concept images in the presentation are credited in Appendix G.

REFERENCES

Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016.

Niyati Bafna, Tianjian Li, Kenton Murray, David R. Mortensen, David Yarowsky, Hale Sirin, et al. The translation barrier hypothesis: Multilingual generation with large language models suffers from implicit translation failure. *arXiv preprint arXiv:2506.22724*, 2025.

Lukas Berglund, Meg Tong, Max Kaufmann, Mikita Balesni, Asa Cooper Stickland, Tomasz Korbak, and Owain Evans. The reversal curse: LLMs trained on "A is B" fail to learn "B is A". In *International Conference on Learning Representations*, 2024.

Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, Soumya Sanyal, Sean Welleck, Xiang Ren, Allyson Ettinger, Zaid Harchaoui, and Yejin Choi. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems*, 2023.

Adam E. Friedman, Stevan Harnad, and Rushen Shi. Tolerance principle and small language model learning. *arXiv preprint arXiv:2601.12179*, 2026.

Lukas Galke, Yoav Ram, and Limor Raviv. Deep neural networks and humans both benefit from compositional language structure. *Nature Communications*, 15, 2024.

Jonathan Herzig and Jonathan Berant. Don't paraphrase, detect! rapid and effective data collection for semantic parsing. In *Proceedings of EMNLP-IJCNLP*, 2019.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Najoung Kim and Tal Linzen. COGS: A compositional generalization challenge based on semantic interpretation. In *Proceedings of EMNLP*, 2020.

Christo Kirov and Ryan Cotterell. Recurrent neural networks in linguistic theory: Revisiting Pinker and Prince (1988) and the past tense debate. *Transactions of the Association for Computational Linguistics*, 6, 2018.

Alina Klerings, Jannik Brinkmann, Daniel Ruffinelli, and Simone Paolo Ponzetto. Steering language models in multi-token generation: A case study on tense and aspect. In *Proceedings of EMNLP*, pp. 8632–8650, 2025.

Brenden Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *International Conference on Machine Learning*, 2018.

Malik Marmonier, Rachel Bawden, and Benoît Sagot. Explicit learning and the LLM in machine translation. In *Proceedings of EMNLP*, 2025.

Alana Marzoev, Samuel Madden, M. Frans Kaashoek, Michael Cafarella, and Jacob Andreas. Unnatural language processing: Bridging the gap between synthetic and natural language data. *arXiv preprint arXiv:2004.13645*, 2020.

Kate McCurdy, Sharon Goldwater, and Adam Lopez. Inflecting when there's no majority: Limitations of encoder-decoder neural networks as cognitive models for German plurals. In *Proceedings of ACL*, 2020.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering Llama 2 via contrastive activation addition. In *Proceedings of ACL*, 2024.

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.

Linlu Qiu, Peter Shaw, Panupong Pasupat, Tianze Shi, Jonathan Herzig, Emily Pitler, Fei Sha, and Kristina Toutanova. Evaluating the impact of model scale for compositional generalization in semantic parsing. In *Proceedings of EMNLP*, 2022.

Kathryn D. Schuler, Charles Yang, and Elissa L. Newport. Testing the tolerance principle: Children form productive rules when it is more computationally efficient to do so. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, 2016.

Garrett Tanzer, Mirac Suzgun, Eline Visser, Dan Jurafsky, and Luke Melas-Kyriazi. A benchmark for learning to translate a new language from one grammar book. In *International Conference on Learning Representations*, 2024.

Tatoeba contributors. Tatoeba: a collection of sentences and translations. `https://tatoeba.org`, 2026.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering language models with activation engineering. *arXiv preprint arXiv:2308.10248*, 2023.

Jannis Vamvas et al. Translation asymmetry in LLMs as a data augmentation factor: A case study for 6 Romansh language varieties. *arXiv preprint arXiv:2603.25489*, 2026.

Yushi Wang, Jonathan Berant, and Percy Liang. Building a semantic parser overnight. In *Proceedings of ACL*, 2015.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Charles Yang. *The Price of Linguistic Productivity: How Children Learn to Break the Rules of Language*. MIT Press, 2016.

A THE INVENTED LANGUAGE

Grammar: Subject–object–verb order, no articles, case endings (object, location and 21 relation cases such as ON, TO, WITH, FROM), every tense an ending, full person/number/gender agreement on the verb, a negation ending, irregular past stems, progressive, perfect and modal endings (CAN, MUST, SHOULD, MAY, WOULD, LET), yes/no questions as an ending, question words left in place, and sound changes (sandhi) where pieces join.

Vocabulary: 11,078 root words (including 2,355 abstract and mass nouns), hand-listed everyday words, and words built from stackable pieces: `fun>ADJ` (funny), `hope>LESS` (hopeless), `run>AGENT` (runner), `happy>NOT>QUAL` (unhappiness); 2,127 such words verified with WordNet, plus compounds such as `book_shop`.

Script: 27 invented glyphs from the Unicode Private Use Area, an abugida like Tamil script (each sign a consonant with an inherent vowel, or a vowel). Without being added to the vocabulary each glyph would be split into three byte tokens; we add each as one new token.

Analysis fields: Each event has optional fields `S` (doer), `V` (verb with endings), `O` (object), `L` (place), `M` (manner), `A` (describing words after "be"), `P` (noun after "be"), `C` (joining word), `I` (greeting), `Y` (person spoken to), `T` (time phrase) and one field per relation (`ON=`, `TO=`, ...); events are separated by ";". Partial inputs (a bare noun phrase, a bare verb) are also translated.

Earlier version: The first language version (4,320 English content words, one event per sentence) was used for the held-out structure study (Figure 1, left) and the script study (Appendix C). The final version keeps every first-version form unchanged (checked on 4,587 clauses).

B TRAINING DETAILS AND COMPUTE

Base model	Qwen3-0.6B-Base (one run: Qwen3-1.7B-Base, 2 epochs)
Adapter	LoRA $r = 16$, $\alpha = 32$, dropout 0; q, k, v, o, gate, up, down projections
New tokens	27 glyphs, trainable tied embeddings, start = embedding of the glyph's sound
Optimiser	AdamW, lr 2×10^{-4}, weight decay 0, 3% warm-up, linear decay
Batch	16×2 gradient accumulation; long batches split by a 1,600-token budget
Data	80,000 translation + 2,500 refusal + 6,000 replay examples; word floor 12
Length	4 epochs, 11,064 steps; fp16; about 6 GPU-hours on a Kaggle T4
Compute	free Kaggle (about 30 GPU-h per account per week), laptop RTX 2050 (4 GB) for scoring, probes, steering

C STAGE 1: THE SCRIPT AND THE TOKENIZER

The project began with a tokenization question: does the number of tokens a script costs predict how well a model learns a language written in it? In a 2×2 grid (English-like vs Tamil-like grammar, Latin vs Tamil script, three random languages per cell, 20k sentences) content-word accuracy was 0.82 (easy, Latin), 0.86 (easy, Tamil script, $3\times$ more tokens), 0.04 (hard, Latin) and 0.24 (hard, Tamil script): more tokens did not mean worse learning, and grammar dominated; at 50k sentences all four cells reach 0.98–0.99. What hurt was *byte pieces that do not line up with known characters*: invented glyphs split into 3 byte tokens collapse at 20k sentences (exact 0.00). Adding each glyph as one token, with its embedding starting at the embedding of its sound, is the method used in all main runs.

D ADDITIONAL RESULTS

Decoding: On Tatoeba, beam search with 4 beams gives meaning exact 0.618 vs 0.617 greedy (5 fixes, 4 breaks, $p = 1.0$) and a majority vote of 5 samples 0.622 ($p = 0.51$): other outputs repeat the same misreading.

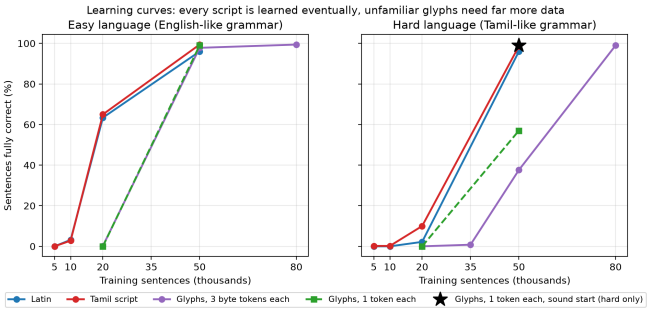


Figure 8: Stage 1 learning curves: content-word accuracy against training sentences for the four grammar × script cells and the invented-glyph runs (first language version).

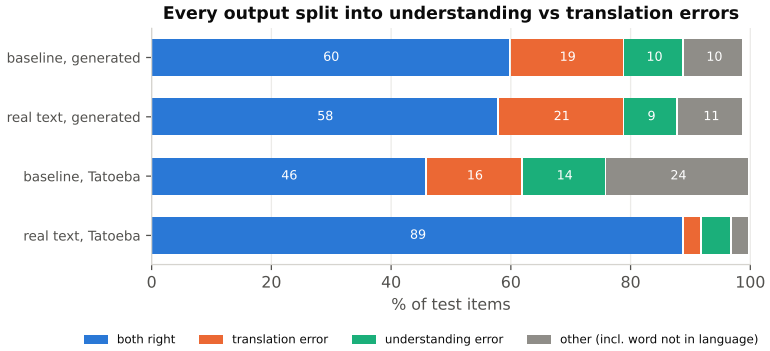


Figure 9: Every output classified by comparing the analysis and the glyphs with the gold (% of items, 1 seed each). "Other" is mostly an analysis using a word the language does not have. Real text removes most errors on Tatoeba but none of the translation errors on generated sentences: what it adds is vocabulary, not a better command of the endings.

What the base model knows: On generated statements the untrained base model answering comprehension questions gets 0.649 of facts right, the instruct version 0.725, and linear probes on the frozen base model recover tense 0.94–0.99, negation 0.99–1.00 and word role 0.87–0.89. The fine-tuned baseline reaches 0.987: the base model encodes much more than it can be prompted to say.

Real English by epoch: The baseline's meaning on Tatoeba is 0.577 / 0.612 / 0.617 after epochs 1 / 2 / 4 and on the sealed set 0.429 / 0.463 / 0.476: reading real English is almost settled by epoch 2; later epochs mainly improve the glyphs.

Why the sealed set is harder than Tatoeba: An automatic totals-only breakdown (predictions written first) finds that on single-clause sentences the baseline scores the same on the sealed set as on Tatoeba (0.61 vs 0.62); the gap comes from multi-clause sentences (sealed 0.17), which are a third of free writing but 1% of Tatoeba. Real text adds +0.15 on single clauses and nothing on multi-clause ones.

E SIDE STUDY: NO GROKKING

Grokking is a late jump in test accuracy long after training accuracy saturates, usually seen with strong weight decay on small algorithmic datasets (Power et al., 2022). If it happened here, held-out combinations might be learned with enough training instead of more data. We trained three small settings far past saturation with weight decay 1.0 (on the LoRA adapter and the new embeddings; the base model is frozen):

Run	Steps	Train	Test (end)	Extra test (end)	Logit norm
translation, 4,000 sentences	12,000	0.93	0.80 (new combinations)	0.49 (new clause structure)	1,570 → 3,130
maths, 40% of the table	60,000	1.00	0.73 (new pairs)	0.11 (3-digit)	1,160 → 5,720
maths, 60% of the table	60,000	0.99	0.84 (new pairs)	0.09 (3-digit)	4,599 → 5,603

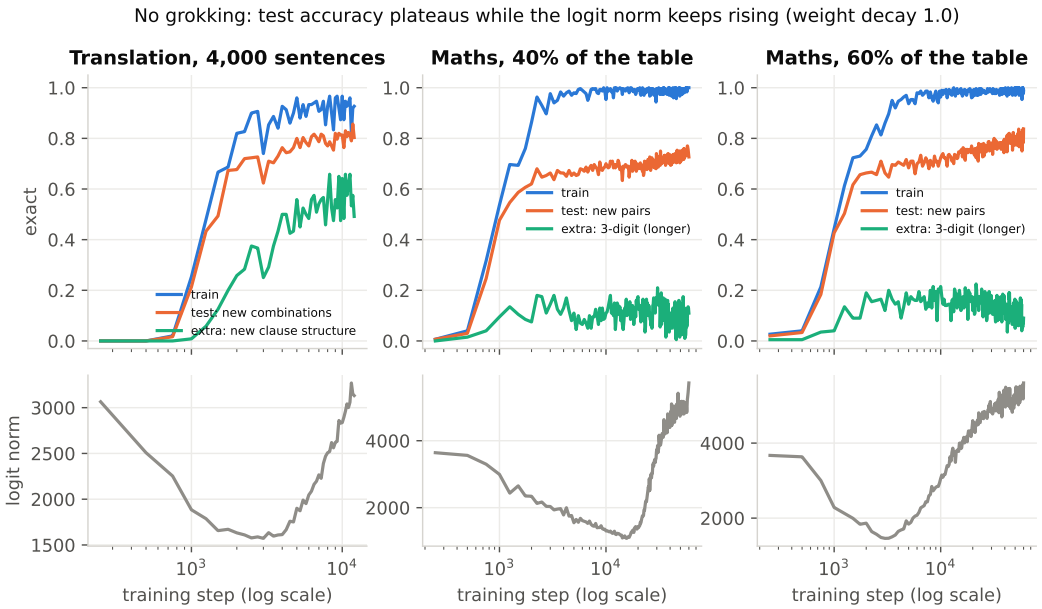


Figure 10: Training curves of the three grokking runs (log step axis). Top: train, test and harder "extra" test accuracy. Bottom: the norm of the output logits.

There is no grokking transition in any of the three (Figure 10). Training accuracy saturates around step 6,000; the translation test is flat from about 4k steps and the 40% maths test stays at 0.71–0.76 from about 6k. With 60% of the table the test score keeps rising slowly (0.74 at 20k steps to 0.84 at 60k), so a later gradual improvement cannot be ruled out, but there is no jump, and the longer 3-digit problems never improve. The logit norm first falls and then rises steadily: the model makes its memorised answers more confident instead of simplifying, the opposite of the grokking signature. Caveat: the 40% run used weight decay 0.1 for its first 16k steps (resumed from a checkpoint), so 40% vs 60% is not a clean comparison.

F Code

The code is submitted with this report and is public at `https://github.com/Lsidvd234/alien-rosetta`. Main parts: `conlang/` (grammar, lexicon, script and the v2 generator), `train/run.py` (data building, glyph tokens and their initialisation, LoRA training, evaluation), `train/maths.py` (base-7 arithmetic and scratchpads), `analysis/` (scoring, significance tests, error split, reader/writer bottleneck, loanword and sound-rule analysis, probes and steering), `kaggle_tools/` (job queues for free Kaggle GPUs), and `sealed/` (the frozen test set is not included).

G Image credits (presentation)

Tamil alphabet chart: public domain, Wikimedia Commons. Decoder-only transformer diagram: Marxav, CC BY-SA 4.0, Wikimedia Commons. LoRA diagram: Figure 1 of Hu et al. (2022). Activation-addition diagram: Figure 1 of Turner et al. (2023).